

# 企业真正缺少的， 是选对 Agent 方案的方法。

场景是活分布，样本持续流入。AgentFit 从最简方案开始，在样本上训练，依据证据更新四层方案，迭代到收敛后交付。

## 用户最小输入

场景材料+样本

评价方式

其他全部可选

工具清单 / 审核人 / 领域知识 / 架构偏好 /  $\lambda$  权重：有则直接映射到对应层，没有则 AgentFit 自动构建。

五种合法交付：全自动 / 部分自动 / 降级 / 保留人工 / 拒绝自动化。

## AGENTFIT · 训练循环

01

### 前向执行

取一批样本，用当前方案执行

02

### 损失归因

自底向上 L1→L4，因果性验证

03

### 反向更新

分层更新建议 + 正则约束

04

### 回归验证

旧样本不遗忘，迭代到收敛

方案不是设计出来的，是训练出来的。

# AgentFit: 方案不是设计出来的，是训练出来的。

传统做法"设计→评审→部署"靠经验，AgentFit 用"训练→评估→更新→收敛→部署"靠数据。

## 传统方式

### 设计 → 评审 → 部署

一次性交付 · 静态架构 · 靠架构师经验  
上线后发现问题 → 人工排查 → 手工修改  
不可量化 · 不可追溯 · 不可维护

## AgentFit

### 训练 → 评估 → 更新 → 收敛 → 部署

迭代训练 · 动态方案 · 靠真实样本数据  
失败样本自底向上归因到具体层 → 修那一层  
可量化 · 可追溯 · 可维护 · 可审计

## 类比机器学习 (Agent↔ML 硬映射)

| ML 概念 | AgentFit 对应                                          | 怎么变          |
|-------|------------------------------------------------------|--------------|
| 模型参数  | L1-L4 各层内容 (方案 = 模型)                                 | 每轮训练后由归因驱动更新 |
| 损失函数  | $(1 - \text{通过率}) + \sum \lambda_i \cdot \text{正则项}$ | 每轮训练后计算      |
| 梯度更新  | 分层更新建议 (diff 可见)                                     | 人审后以原子事务应用   |
| 过拟合检测 | 回归池重跑 (旧样本)                                          | 遗忘即回滚        |

# 四层骨架：Solid → 能力 → 知识 → 拓扑

每一层只做一件事。层间逐层调用，同层禁止执行时互调。

## L4 · 行为拓扑层

Agent 架构 + 协作模式 + 行为序列 + 人工介入位置

多 Agent 协同 + Agent 间上下文传递 · Simple First 起步 · 更新：低（需证据）

## L3 · 可复用知识层

Skill + 路由规则 + 排查链 + 决策阈值 + 经验记录

任务拆解 = 排查链 / Skill 步骤序列 · 更新：高 · Memory = 经验记录

## L2 · 可二次开发能力层

对 Solid 的安全封装 + 组合 + 口径统一 + 送审路由

工具调用统一入口：safe\_\* 封装 · 组合 · 口径 · 送审 · 更新：中

## L1 · Solid 层（固定地基）

固定原子能力：API + 数据库 + 知识库 + 外部系统 + 人工审核

更新频率：最低（只增不删）· human\_review 与 query\_database 同级

铁律：L3 不能直接调 L1 · 同层禁止执行时互调 · 每个能力必须追溯到物理基础设施

# 存在依赖：每个能力追溯到物理基础设施。

不能凭空创造。任何一层声明能力，下层必须已提供支撑。

## 声明链（自顶向下验证）

### L4 "我有诊断 Agent"

- L3 必须已有 L3 "诊断排查链"
- L2 必须已有 L2 "safe\_get\_device\_state"
- L1 必须已有 L1 "get\_device\_state 原子"
- 基础设施已有 设备管理系统 API

任何一环断了 → 不能在上层创建 → 更新目标下移到断的那层，全链补齐再建

## ChangeTransaction（原子事务）

级联更新必须原子执行：

- ① BEGIN — 记录当前方案快照
- ② APPLY — 自底向上（L1→L2→L3→L4）
- ③ VALIDATE — 存在依赖完整性检查
- ④ COMMIT — 通过 → 版本号 +1
- ⑤ ROLLBACK — 失败 → 恢复快照

禁止中间状态 · 禁止部分成功 · 要么全成功要么全回滚

保证：无幻影能力 · 全链路可追溯 · 天然可维护 · 无技术债 · 部署即生效

# 训练循环：执行 → 归因 → 更新 → 回归 → 迭代。

九步闭环。每步产出结构化数据，训练日志记录一切。

①

## 前向执行

取一批样本用当前方案执行

②

## 损失归因

自底向上L1→L4 + 因果性验证

③

## 聚合分析

损失模式 + 正则指标计算

④

## 更新建议

分层建议 +  $\lambda$  调节

⑤⑥

## 人审

审核方向和安全边界

## 回归验证 (⑧) + 训练日志 (⑨)

**回归：**旧样本重跑，确认更新没破坏已有能力。通过率 < 95% → 自动回滚。

**日志：**每轮追加通过率、损失分布、更新内容、回归结果、 $\lambda$  变化、成本 — 哈希链保护。

## 收敛条件

通过率连续 N 轮无显著提升 · 剩余失败全部需人工 · 预算/时间超限 · 回归池通过率下降

# 自底向上归因：错在哪层、为什么、怎么修。

从 L1 开始逐层检查。找到异常后验证因果性，不是"找到即停"。

## 归因流程（自底向上）

### Step 1 → L1 "原子存在吗？"

缺 → 验证因果 → 是根因则停 / 否则标记附带问题

### Step 2 → L2 "封装正确吗？"

错 → 在关键路径上？是 → 停 / 否 → 附带问题

### Step 3 → L3 "路由对了吗？"

走错分支 → 验证修复后能通过？ → 停

### Step 4 → L4 "架构适合吗？"

不适合 → 停 / 适合 → 需要人工 or 评价标准有误

## 因果性验证（反事实推演）

发现异常 ≠ 导致失败。

验证方法："如果这个异常被修复，样本能通过吗？"

例：L2 有日志格式问题（旁路操作），但真正失败原因是 L3 路由错误。验证因果性后跳过 L2 日志问题，正确归因到 L3。

**附带问题：**不构成根因的异常记录在案，后续训练中可能升级为根因。

就近归因 · 逐层排查 · 找到即验证 · 附带问题不阻塞

# 正则约束：可维护性从口号变成数字

16 指标 × 4 层。每层正则 =  $\max(0, \text{worst\_violation\_ratio})$ 。λ 两级调节。

## 聚合公式

$$\begin{aligned} Lx\_reg &= \max(0, \text{worst\_violation}) \\ \text{总损失} &= (1 - \text{通过率}) \\ &\quad + \sum \lambda_i \cdot Li\_reg \end{aligned}$$

结构正则=方案验证器静态算 · 行为正则=归因器每轮从轨迹算

## λ 默认值（上层惩罚更重）

$\lambda_1=0.1$  原子耦合 ·  $\lambda_2=0.2$  工具复杂

$\lambda_3=0.3$  知识集中 ·  $\lambda_4=0.4$  拓扑复杂

Level 1: 自动  $\leq \pm 20\%$  · Level 2: 人审  $> \pm 20\%$

$\lambda_4 > \lambda_3 > \lambda_2 > \lambda_1$ ：越上层过度复杂越危险，惩罚越重

## 关键指标示例（16 项中）

L1 原子使用率 > 80% → 一个原子做所有事 → 拆分建议

L3 链路覆盖度 > 60% → 万能路由 →  $\lambda_3$  连续 2 轮超限自动 ×1.2

L4 人工介入率 > 30% → 自动化价值低 → 重新评估范围

# 可审计：训练日志（哈希链保护）

每轮追加不可篡改记录。任何人可回答"为什么改、改了什么、有没有改坏"。

训练日志条目（每轮一条，previous\_hash 串成链）

```
{ "epoch": 3, "pass_rate": 0.85,  
  "loss": {"L1":0,"L2":1,"L3":4,"L4":0,"human":5},  
  "updates": [{"layer":"L3","change":"compound routing","reason":"3 samples failed"}],  
  "regression": {"tested":87,"passed":87}, "cost": {"usd":2.50},  
  "previous_hash": "abc123..." }
```

- ✓ 可回答"为什么改" → 3个样本在复合退款上失败
- ✓ 可回答"有没有改坏" → 回归 87/87 通过
- ✓ 可回答"总成本" → 3轮 \$2.50



# 可追溯：失败 → 根因 → 修复 → 验证

每个失败样本从 L1 到 L4 全链路归因，附带因果性验证。

## 损失轨迹示例 (LossTrace)

样本 #42 失败 (复合根因：飞行模式 + 漫游)

L1: 原子存在 ✓ → 跨过

L2: 封装正确 ✓ → 跨过

L3: 路由只处理了漫游 (主因)，没处理飞行模式 (次因)

因果性验证: "修复路由后能通过吗?" → 是 → **确认根因 = L3**

附带问题: L2 日志格式 (旁路, 不阻塞) → 记录在案

从失败样本 → 具体层的具体规则的具体分支 → 全链路可追溯

# 可量化：通过率 / 成本 / 正则值 / 回归率

每轮训练自动产出全部指标。数字说话，不靠感觉。

85%

通过率

训练 3 轮后

\$0.006

成本/样本

vs 人工 \$15

100%

回归通过率

必须 100%

0.08

L3 正则值

阈值 0.10 内

各层损失分布（第 3 轮）— 归因给哪层，一目了然

L1  0 (0%)

L2  1 (4%)

L3  4 (16%)

人工  5 (20%)

解读：主要损失在 L3 路由 → 本轮更新集中在 L3；20% 需人工 → 交付形态选“部分自动”的证据

# Telecom 实测：80% baseline → AgentFit 目标

$\tau^2$ -bench telecom 基准已跑通，10 样本实测通过率 80%，成本 \$0.006/样本。

## Baseline (裸跑 · 已实测)

DeepSeek +  $\tau^2$ -bench 直接执行

10 样本 · 8 通过 · 80%

成本: \$0.006/样本 · 2 个失败无归因

## AgentFit (训练后 · 目标)

四层方案 + 训练循环 + 归因更新

目标:  $\geq 80\%$  通过率 + 四可保证

额外交付: 训练记录 + 适用边界 + 持续监控

## Baseline 不具备的能力

✗ 不知道为什么失败 (无归因)    ✗ 不知道能不能维护 (无正则)

✗ 改了会不会改坏 (无回归)    ✗ 不知道适用边界 (无收敛分析)

→ AgentFit 的价值 = 不只是通过率，而是 baseline 不具备的四可保证；系统自身即五角色多 Agent 协同，验证/审计为确定性代码

# 五种合法交付：包括"拒绝自动化"

"不自动化"和"自动化"一样是合法结论，且有证据支撑。

## 全自动

AI 全权处理  
通过率  $\geq$  阈值  
风险可控

## 部分自动

AI 常规 + 人关键  
多数自动  
少数需人

## 降级

简化版 + 条件限制  
预算/能力有限

## 保留人工

不值得自动化  
成本 > 人工

## 拒绝自动化

不该用 AI  
通过率太低  
风险太大

## 交付物

-  可部署方案包 +  训练记录（通过率曲线/更新日志/回归验证）
-  适用边界（能自动/需人工/为什么）+  持续监控（漂移检测/重训练触发）

AgentFit 不预设 AI 总是更好。Fit = 有证据地选对方案。

# 四层骨架完整定义

完整版见 docs/agentfit-skeleton.md (仓库开源) · 本页为各层职责速览

## L4 · 行为拓扑层

要素：Agent 数量 · 协作模式 · 行为序列 · 人工介入位置 · 升级路径  
多 Agent 协同与 Agent 间上下文传递在本层定义 · Simple First · 改拓扑需证据 + 人审

## L3 · 可复用知识层

五类知识严格区分：Skill (操作模板) · 路由规则 (IF-THEN) · 排查链 (任务拆解为有序步骤) · 决策阈值 · 经验记录  
Memory = L3 经验记录，不是独立系统 · 上限 200 条，去重/过时/压缩治理

## L2 · 可二次开发能力层

四类封装：安全包装 (safe\_\*) · 组合能力 (跨原子上下文传递) · 口径封装 · 送审路由  
工具调用统一经 L2 触达 L1 · 例：safe\_create\_refund = 验证身份 → 检查金额 → 超限送审

## L1 · Solid 层 (固定地基)

原子能力：API + 数据库 + 知识库 + 外部系统 + 人工审核 (human\_review 与 query\_database 同级)  
输入 question/context/options/domain → 输出 decision/reason/conditions

## 三组跨层约束

**纵向：** L4→L3→L2→L1 逐层调用，不跨层 (唯一例外：L4 直调 L2 人工审批)  
**横向：** 同层禁止执行时互调 (L3 路由规则分发到 Skill 是组织功能，合法；Skill 步骤调 Skill 是执行耦合，禁止)  
**存在：** 每层能力必须追溯到物理基础设施 · 构建强制自底向上 · 验证规则见 04 页

# 反向归因算法（伪代码）

四步逐层检查 + 因果性验证（反事实推演）· 完整版见 docs/agentfit-skeleton.md §二

## 主流程: `attribution(sample)`

```
for layer in [L1, L2, L3, L4]:
    anomaly = check(layer, trace)
    if anomaly is None: continue # 这层没问题, 往上查
    if is_root_cause(anomaly, trace):
        return root_cause=layer, stop
    else:
        side_issues.append(anomaly) # 附带问题, 不阻塞
return "needs_human" # 四层都没问题

# 附带问题 → 聚合阶段统一报告
# 主因修复后可能升级为根因（浮现）
```

## 因果性验证: `is_root_cause()`

反事实推演: "如果这个异常被修复, 样本能通过吗?"

`step = find_step(trace, anomaly)`

- ① 异常不在轨迹中 → False
- ② 无下游依赖（旁路，如日志）→ False，记附带问题
- ③ 下游依赖且输出会影响结果 → True，候选根因

效果: L2 日志格式问题（旁路）不会挡住真正的 L3 路由错误

自底向上 · 找到即验证 · 就近归因（问题在哪层归因给哪层，不往上推）· 附带问题不阻塞

# 正则指标完整清单（16 项）

每个指标 = 三元组（数据源, 计算时机, 计算者）· 完整操作定义见 docs/agentfit-skeleton.md §三

## 结构性正则（静态 · 方案更新后 · 方案验证器算）

| 指标          | 层  | 阈值             |
|-------------|----|----------------|
| 原子使用率       | L1 | >80% 过耦合       |
| 原子稀缺率       | L1 | >30% 冗余        |
| 封装复杂度       | L2 | >5 原子太复杂       |
| 工具复用率       | L2 | <2 链路低复用       |
| 链路长度 / 分支因子 | L3 | >10 步 / >15 分支 |
| 经验引用率       | L3 | >30% 未引用       |
| Agent 数量    | L4 | >5 需要理由        |

## 行为性正则（动态 · 每轮训练后 · 归因器从轨迹算）

| 指标            | 层  | 阈值         |
|---------------|----|------------|
| 原子增长率         | L1 | >5 个/轮过快   |
| 审批延迟          | L2 | >5min 瓶颈   |
| 链路覆盖度         | L3 | >60% 过集中   |
| 知识冲突率 / 经验去重度 | L3 | >5% / >20% |
| 人工介入率         | L4 | >30% 价值低   |
| 通信开销（上下文传递）   | L4 | >10 轮/样本   |
| 拓扑变更频率        | L4 | >1 次/轮不稳   |

## λ 两级调节（防“单点耦合爆炸”）

**Level 1 自动：**单  $\lambda \leq \pm 20\%$ ，对应层正则连续 2 轮超阈值自动调，仅记日志 · 每轮最多调 1 个，累计  $\leq \pm 50\%$

**Level 2 人审：**单  $\lambda > \pm 20\%$  或多  $\lambda$  同调 · 生成结构化建议（触发/当前值/建议值/预期影响/权衡）进入审队列 · 用户不响应 = 不应用

用户场景：小团队  $\lambda$  全×2 宁简单 · 合规严格  $\lambda_2$  调大要透明 · 极致通过率  $\lambda$  调小容过拟合

# 测试场景执行方案（Telecom）

五阶段全链路：环境 → 样本 → 初始方案 → 训练循环 → 对比交付 · 详见 docs/test-scenario.md

## ① 环境准备

$\tau^2$ -bench telecom 环境  
DeepSeek API key  
AgentTeams 运行底座  
成本监控就位

## ② 样本准备

500 样本分层抽样  
训练/回归/测试 = 6:2:2  
按故障类型分层  
难度标签 A-D 四组

## ③ 初始方案

最简起步（Simple First）  
单 Agent + 基础工具  
从样本归纳 L3 初始链路  
入用默认值

## ④ 训练循环

九步闭环迭代  
每轮 50 样本  
归因→更新→回归  
收敛即停（预计 8-10 轮）

## ⑤ 对比交付

A/B/C/D 四组对照  
baseline vs 训练后  
产出四可报告  
交付方案包+边界

## 预算估算（全流程）

500 样本  $\times$  \$0.006 = \$3.00（一轮）  
训练+回归+测试  $\approx$  **\$7.05 总预算** · 超支自动熔断

## 已完成的实测（不是空想）

$\tau^2$ -bench telecom 10 样本已跑通：**80% 通过率** · \$0.006/样本  
AgentTeams 平台 8 轮真实运行（R1-R8）已完成

## 成功标准

通过率  $\geq$  baseline 且正则全绿 · 回归池 100% 不遗忘 · 每个失败样本有 LossTrace · 交付五选一结论有证据 · 总成本 < \$10



# 开源承诺与诚实披露

MIT License · [github.com/kailiangshang/agentfit](https://github.com/kailiangshang/agentfit) · 设计全公开，未实现部分如实列出

## 开源范围 (MIT)

四层骨架文档 (agentfit-skeleton.md) · 训练循环代码 · 测试场景与执行方案 · 训练日志与回归池数据  
骨架定稿后不再变更，实现驱动迭代

## 依赖清单

| 依赖              | 角色         | License |
|-----------------|------------|---------|
| $\tau^2$ -bench | 执行+评测环境    | MIT     |
| DeepSeek API    | LLM 提供者    | 商业 API  |
| AgentTeams      | Agent 运行底座 | 官方许可    |

## 未实现披露 (如实列出，不装完成)

① 16 个正则指标有操作定义，未代码化 ② ChangeTransaction 有伪代码，未实现 ③ 漂移检测有设计，未部署验证  
④ 持续监控有配置定义，未运行 ⑤ 仅 telecom 场景有实测数据 (80% baseline)，其他场景未测试  
为什么敢交：骨架与算法设计完备 (6 轮审计定稿) · 评测环境已跑通 · 未实现项全部有明确设计文档和实现路径

数据授权： $\tau^2$ -bench 公开基准 (MIT) · 训练数据由 AgentFit 生成并开源 · 不使用任何私有企业数据